

A PROJECT REPORT ON

QUERY BASED VIDEO SUMMARISER

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Jayesh Kulkarni
Rabiya Javed Farooq
Shivam Gaikwad
Vartika Mishra

Exam No: B400050460
Exam No: B400050559
Exam No: B400050391
Exam No: B400050626



DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025



CERTIFICATE

This is to certify that the project report entitled

QUERY BASED VIDEO SUMMARISER

Submitted by

Jayesh Kulkarni	Exam No: B400050460
Rabiya Javed Farooq	Exam No: B400050559
Shivam Gaikwad	Exam No: B400050391
Vartika Mishra	Exam No: B400050626

are bonafide students of this institute and the work has been carried out by them under the supervision of **Prof. V.V. Bagade** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Prof. V.V. Bagade
Internal Guide
Dept. of Computer Engg.

Dr. G.V. Kale
Head
Dept. of Computer Engg.

Dr. S.T. Gandhe
Principal
Pune Institute of Computer Technology

Place:Pune
Date:08-05-2025

ACKNOWLEDGEMENT

*It gives us great pleasure in presenting the project report on ‘**QUERY BASED VIDEO SUMMARISER**’.*

*We would like to take this opportunity to thank **Prof. V.V. Bagade** giving me all the help and guidance we needed. We are really grateful to them for their kind support. Their valuable suggestions were very helpful.*

*We are also grateful to **Dr G.V. Kale**, Head of Computer Engineering Department, PICT for his indispensable support, suggestions.*

Finally, I would like to acknowledge the authors of the various research papers and articles that I referred to, whose work significantly contributed to the successful completion of this project.

Jayesh Kulkarni
Rabiya Javed Farooq
Shivam Gaikwad
Vartika Mishra
(B.E. Computer Engg.)

ABSTRACT

In the current digital age, video content has become one of the most prevalent mediums for communication, learning, and information dissemination. With platforms hosting an overwhelming volume of video data, users often face challenges in locating specific information efficiently. Traditional video summarization techniques typically produce a static, one-size-fits-all summary, which does not cater to the varying information needs of individual users. As a result, users are left sifting through large portions of irrelevant content, leading to information overload and decreased satisfaction. Traditional video summarization techniques, while helpful to some extent, are largely static in nature. They generate generic summaries based on fixed criteria such as shot changes, visual saliency, or audio cues, resulting in a one-size-fits-all output. These methods fail to accommodate the diverse and context-specific information needs of individual users. Consequently, users are forced to sift through large portions of irrelevant or redundant content in search of the specific segments they require. This process not only leads to information overload but also causes frustration, reduced engagement, and diminished user satisfaction.

Contents

1	Introduction	2
1.1	Overview	3
1.2	Motivation	4
1.3	Problem Definition and Objectives	4
1.4	Project Scope & Limitations	5
1.5	Methodologies of Problem solving	6
2	Literature Survey	7
3	Software Requirements Specification	10
3.1	Assumptions and Dependencies	11
3.2	Functional Requirements	11
3.2.1	System Feature 1(Query Interpretation)	11
3.2.2	System Feature2 (Video Content Analysis and Segmentation)	11
3.2.3	System Feature 3 (Summary Generation and Output Presentation)	11
3.3	External Interface Requirements	12
3.3.1	Software Interfaces	12
3.3.2	Communication Interfaces	12
3.4	Nonfunctional Requirements	13
3.4.1	Performance Requirements	13
3.4.2	Safety Requirements	13
3.4.3	Security Requirements	13
3.4.4	Software Quality Attributes	13
3.5	System Requirements	14
3.5.1	Database Requirements	14
3.5.2	Software Requirements (Platform Choice)	14
3.5.3	Hardware Requirements	14
3.6	Analysis Models: Query-Based Video Summarizer	15
3.7	System Architecture	18

3.8	Mathematical Model	19
3.9	Data Flow Diagram	22
3.10	UML Diagrams	23
3.11	Sequence Diagram	24
4	Project Plan	26
4.1	Project Estimate	26
4.1.1	Reconciled Estimates	26
4.1.2	Project Resources	26
4.2	Risk Management	28
4.2.1	Risk Identification	28
4.2.2	Risk Analysis	28
4.2.3	Overview of Risk Mitigation, Monitoring, Management	28
4.3	Project Schedule	29
4.3.1	Project Task Set	29
4.3.2	Task Network	29
4.3.3	Timeline Chart	29
4.4	Team Organization	30
4.4.1	Team structure	30
4.4.2	Management reporting and communication	30
4.5	QA Test Cases	31
5	Project Implementation	35
5.1	Overview of Project Modules	35
5.2	Tools and Technologies Used	36
5.3	Algorithm Details	37
5.3.1	Algorithm 1: Video Upload and Audio Extraction . . .	37
5.3.2	Algorithm 2: Audio Transcription Using Speech Recognition	38
5.3.3	Algorithm 3: Transcript Tokenization and Semantic Encoding	38
5.3.4	Algorithm 4: Query Answering Using Contextual Similarity and Generative Model	39
5.3.5	Algorithm 5: Grammar Correction of Generated Answer	40
6	Software Testing	41
6.1	Type of Testing	41
6.2	Test Cases and Results	42

7	Results	44
7.1	Outcomes	44
7.2	Screen Shots	44
8	Conclusions	46
8.1	Conclusions	46
8.2	Future Work	46
8.3	Applications	47
9	Appendix	49
9.1	Appendix A	49
9.2	Appendix B	51
9.2.1	Survey Paper Details:	51
9.2.2	Implementation Paper Details:	51
9.3	Appendix C: Plagiarism Report	52
10	References	53

List of Figures

3.1	System Architecture Diagram	18
3.2	Mathematical Model Diagram	19
3.3	Overview of Mathematical Model	20
3.4	Data Flow Diagram	22
3.5	Activity Diagram	23
3.6	Sequence Diagram	24
4.1	An example image	31
4.2	An example image	32
4.3	An example image	32
4.4	Before upload	33
4.5	After upload	33
7.1	Screenshot	44
7.2	Screenshot	45
7.3	Screenshot	45
7.4	Screenshot	45
7.5	Screenshot	45
9.1	Plagiarism Report of the Project Report	52

LIST OF ABBREVIATIONS

Abbreviation	Illustration
ML	Machine learning
NLP	Natural Language Processing
QBS	Query-Based Summarization
IR	Information Retrieval
ASR	Automatic Speech Recognition
RNN	Recurrent Neural Network
CNN	Convolutional Neural Network
VQA	Video Question Answering
CV	Computer Vision

CHAPTER 1

INTRODUCTION

1.1 Overview

In the era of digital media, video data has become ubiquitous and deeply integrated into our daily lives. Whether it's in the form of educational content, entertainment, tutorials, surveillance footage, or news broadcasts, video content dominates modern digital communication and information dissemination. This explosive growth of video material has made it one of the most widely consumed and generated media formats globally.

However, most raw video footage tends to be excessively long, often containing repetitive or irrelevant content. Consequently, viewers are frequently burdened with the task of sifting through lengthy videos to find specific, useful information. This process is not only time-consuming but also inefficient, especially when the information required is buried deep within hours of content. The overwhelming volume of available video data has therefore created a pressing need for efficient techniques that can help users explore and navigate video content more effectively.

Traditional video summarization techniques attempt to address this problem by condensing long videos into shorter summaries. However, these approaches typically generate a single, static summary for a video, regardless of the unique interests or requirements of individual users. This one-size-fits-all model fails to consider the diverse and dynamic nature of user information needs. As a result, users often still have to watch non-relevant parts of the summary to locate the information they truly care about, which undermines the purpose of summarization.

To overcome these limitations, there is a growing demand for intelligent, adaptive video summarization systems that are capable of generating personalized summaries based on specific user queries. Query-based video summarization represents a significant step forward in this domain. Unlike traditional methods, these systems utilize NLP and content analysis to interpret a user's textual query and extract only the most relevant video segments. This allows for the creation of customized summaries that directly address the user's interests, minimizing cognitive load and maximizing efficiency.

Video summarization, when guided by user queries, can drastically simplify the process of retrieving targeted information from large volumes of multimedia content. By dynamically adjusting to the user's intent, such systems enhance both accessibility and utility of video archives. They not only save time but also improve the precision and relevance of information delivery.

Hence, in our project, we propose a novel method that accepts a natural language query as input and produces a concise video summary tailored

to that query. This system is designed to automatically detect and compile relevant portions of a video, enabling users to gain quick insights without having to navigate through unnecessary material. By aligning video summarization with user intent, our approach significantly advances the effectiveness of video exploration and information retrieval in the digital age.

1.2 Motivation

The ever-growing volume of video content in the digital media era has transformed how people consume information, learn, and communicate. However, this explosion of content has also introduced a significant challenge—users often spend excessive time navigating through lengthy videos to locate specific pieces of information. Traditional video summarization techniques, which generate fixed and generic summaries, fall short in addressing the diverse and dynamic needs of individual users. These static methods ignore the user's intent and context, often leading to irrelevant summaries that waste time and reduce engagement.

This work is driven by the need to overcome these limitations by introducing a flexible, intelligent, and user-centric approach to video summarization. Such a personalized experience not only saves time but also enhances the value and accessibility of video content across various domains—education, journalism, entertainment, and more. Our goal is to revolutionize how users interact with video data, making the exploration process significantly faster, more intuitive, and far more effective.

1.3 Problem Definition and Objectives

Create a video summarizer that extracts relevant content based on user queries, analyze video segments using NLP and video analysis, and provide concise snippets, enabling users to quickly access specific information without watching the entire video. Provide concise and relevant video summaries based on user queries, reducing the need to watch the entire videos. By offering focused summaries, reduce the time users spend searching through lengthy videos, enabling faster access to relevant content. Makes it easier for users to desired information without sifting through entire videos.

1.4 Project Scope & Limitations

This project aims to develop a query-controllable video summarization system that enables users to extract specific information from videos based on their personal queries. Unlike traditional summarization techniques that generate static, generalized summaries, this system focuses on delivering dynamic, personalized content tailored to individual user needs. By integrating Natural Language Processing (NLP), the system can interpret user queries and analyze video transcripts or subtitles to identify the most relevant segments. Additionally, video analysis techniques will be applied to locate and extract corresponding visual and audio snippets, creating concise and meaningful summaries.

The system will include a user-friendly interface that allows users to input queries and instantly view customized video highlights. This approach is intended to improve the efficiency of video navigation, reduce time spent searching for information, and enhance overall user engagement. The project also aims to evaluate the effectiveness of the generated summaries based on user satisfaction, accuracy of content relevance, and time saved compared to manually watching entire videos.

While the system introduces an innovative solution to video summarization, it does have several limitations. First and foremost, the accuracy of the summarization is highly dependent on the quality and availability of video transcripts or subtitles. Videos lacking proper textual metadata may result in poor performance or irrelevant summaries. Additionally, the initial version of the system may be optimized for structured video content such as lectures, tutorials, or news broadcasts, and may struggle with unstructured or entertainment-oriented content like music videos or vlogs.

Another limitation is the system's ability to handle complex or ambiguous user queries. NLP models may find it difficult to interpret queries that are context-sensitive, metaphorical, or vague, leading to suboptimal summary generation. The computational requirements for processing long videos and generating real-time summaries may also pose challenges, especially in resource-constrained environments. Furthermore, the system's understanding of the visual content is limited to basic analysis, and it may not fully capture deeper visual context or emotions expressed solely through visuals. Lastly, language support may be restricted to English or a few commonly spoken languages, limiting accessibility for users seeking summaries in other languages.

1.5 Methodologies of Problem solving

The development of the query-controllable video summarization system follows a structured and multi-stage methodology that integrates natural language processing and video analysis techniques to extract user-relevant content effectively. The process begins with video preprocessing, where input videos are segmented, and relevant metadata such as transcripts, subtitles, or audio is extracted and converted to text using speech-to-text tools if necessary. This textual data forms the foundation for understanding the content of the video.

In the query analysis phase, the system uses NLP techniques to interpret and process the user's query. This involves cleaning the query, performing tokenization, named entity recognition (NER), and extracting keywords or topics using methods such as TF-IDF or word embeddings (like Word2Vec or BERT). These keywords are then matched semantically with the video transcript to identify sentences or segments that closely align with the user's intent.

Next, the relevance matching and segment extraction stage maps the identified textual matches back to their corresponding timestamps in the video. This allows the system to isolate the relevant video snippets that contain or relate to the user's query. Additional filters based on visual scene change detection or audio features can be applied to ensure cleaner and more coherent clip boundaries.

After relevant segments are identified, the snippet generation and summarization phase stitches together the selected clips into a concise, coherent video summary.

Finally, the user interface and evaluation phase focuses on delivering the output to the user in a clean, interactive format. Users can enter new queries, replay summaries, or provide feedback.

CHAPTER 2

LITERATURE SURVEY

1.Video Summarization Techniques: Extractive and Abstractive Approaches

Authors: Huang et al., 2020

The development of effective video summarization techniques has gained significant attention in recent years due to the exponential growth of video content. Extractive techniques focus on identifying and selecting key segments from videos, often using algorithms such as K-means clustering for keyframe extraction and temporal segmentation methods that analyze motion and scene transitions. Conversely, abstractive summarization seeks to generate new content that conveys the video's core message, leveraging advanced models like transformers to capture contextual nuances.

2.Leveraging NLP for Query Understanding in Video Summarization

Authors: Huang & Wu, 2021

Natural language processing (NLP) plays a crucial role in enhancing video summarization systems, particularly in understanding user queries. Techniques such as intent recognition, utilizing models like BERT, enable systems to decipher the underlying meaning of user requests effectively. Additionally, semantic analysis methods allow for the extraction of keywords and phrases that align with user inquiries, facilitating a more targeted approach to summarization.

3.Multimodal Video Analysis for Enhanced Summarization

Authors: Hannun et al., 2014

Video content analysis contributes significantly to summarization techniques. Deep learning models are employed for object and action recognition within video segments, allowing for the identification of key elements that may be relevant to user queries. Furthermore, integrating audio analysis through automatic speech recognition (ASR) enables the incorporation of spoken content, providing a comprehensive understanding of the video.

4.Efficient Information Retrieval for Video Query Systems

Authors: Zhao et al., 2022

Efficient information retrieval is another critical aspect of developing a video summarizer. Implementing advanced indexing and querying methods is essential for rapid retrieval of video segments in response to user queries. Techniques like inverted indexing, traditionally used in text retrieval, can be adapted for video content to improve efficiency. Additionally, incorporating a user feedback loop allows the system to learn from interactions, progressively refining its summarization capabilities based on user preferences.

CHAPTER 3
SOFTWARE REQUIREMENTS
SPECIFICATION

3.1 Assumptions and Dependencies

The successful implementation of the query-controllable video summarization system is based on several underlying assumptions and external dependencies that must be considered throughout the development lifecycle.

It is assumed that the input video content is accompanied by accurate and comprehensive metadata such as subtitles or transcripts, or that automatic speech recognition (ASR) tools can effectively generate such transcripts with acceptable accuracy. The system also assumes that user queries are well-formed and semantically interpretable by the natural language processing (NLP) models employed. Furthermore, it is assumed that the underlying NLP models (e.g., BERT) and video processing libraries are capable of handling contextual matching between video content and user intent effectively.

3.2 Functional Requirements

3.2.1 System Feature 1(Query Interpretation)

The system shall allow users to enter a natural language query that describes the specific information they are seeking from a video. The system must then process this input using Natural Language Processing (NLP) to extract the intent and key concepts of the query.

3.2.2 System Feature2 (Video Content Analysis and Segmentation)

The system shall analyze the video using both textual (transcript) and visual/audio data to identify and extract segments that are relevant to the user query. It must apply video processing techniques to determine the best clip boundaries and relevance.

3.2.3 System Feature 3 (Summary Generation and Output Presentation)

The system shall generate a coherent video summary by combining the relevant video segments and present it in a user-friendly interface. It should ensure smooth transitions and playback functionality for user evaluation.

3.3 External Interface Requirements

3.3.0.1 User Interfaces

- The application should have a user-friendly interface that allows users to input search queries, view relevant video segments, and navigate through the video content.
- A playback option should be provided to preview the extracted video snippets.

3.3.0.2 Hardware Interfaces

- The system must support video input via local file upload or cloud storage options.
- If applicable, integration with external devices such as cameras or recording systems should be possible.
- The system should be compatible with hardware-accelerated video processing units (e.g., GPU-enabled environments) to optimize performance.
- The application must support standard video formats and interfaces (e.g., HDMI, USB) for broader compatibility.

3.3.1 Software Interfaces

1. Operating System: The system should be compatible with commonly used operating systems such as Windows, Linux, or macOS.
2. Jupyter Notebook
3. The system should support integration with publicly available video datasets or custom video uploads for summarization.
4. Programming Language: Python

3.3.2 Communication Interfaces

The system will support HTTP/HTTPS protocols for client-server communication. It may also integrate with cloud APIs or RESTful services for video storage and processing.

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

- The system must process video queries and return relevant snippets within a few seconds, depending on video length and query complexity.
- Video snippet playback should not experience buffering or delays, assuming stable internet connectivity.

3.4.2 Safety Requirements

- The system must prevent data corruption or loss during video upload and analysis.
- Video content should be backed up to prevent accidental deletion during processing.

3.4.3 Security Requirements

- The system must secure user-uploaded videos and ensure that only authorized users can access them.
- All sensitive data, including user queries and video content, must be encrypted in transit and at rest.
- Access control mechanisms should be in place to ensure privacy and data security

3.4.4 Software Quality Attributes

- **Usability:** The system will offer an intuitive and user-friendly interface that allows users to input queries, view summaries, and navigate video content with ease.
- **Performance:** The system will provide quick response times when generating video summaries and retrieving relevant segments, even for large video files.
- **Scalability:** The architecture will support processing multiple and large video files, with the potential to scale using cloud-based solutions.

- **Maintainability:** The codebase will follow modular design principles, making it easier to update NLP models or video processing components in the future.
- **Reliability:** The system will ensure accurate summarization based on user queries and gracefully handle edge cases such as poor video/audio quality or vague queries.
- **Portability:** The application will be designed to run across multiple platforms (Windows, Linux, macOS) without significant modification.
- **Security:** The system will ensure that uploaded videos and query data are handled securely, especially when integrated with cloud storage or external APIs.

3.5 System Requirements

3.5.1 Database Requirements

- A database is required to store metadata about videos (e.g., file name, duration, format), user queries, and the results of processed videos.
- The database should support efficient querying and retrieval of large video metadata files and processed summaries.

3.5.2 Software Requirements (Platform Choice)

- Operating Systems: Windows, Linux, or macOS.
- Frameworks: The backend should use Node.js or Python with video and NLP libraries like OpenCV, FFmpeg, TensorFlow, or spaCy.
- CloudServices: AWS, Google Cloud, or Azure for video storage and scalable processing.

3.5.3 Hardware Requirements

- The system should be able to run on machines with at least 8 GB of RAM and a multi-core processor (4 cores or higher) to handle video processing.
- For real-time processing or large datasets, GPU support is recommended.

3.6 Analysis Models: Query-Based Video Summarizer

1. User Query Understanding Module (NLP)

- **Input:** User's natural language query (e.g., "Show me scenes with people dancing")
- **Techniques:**
 - **Intent Detection:** Use models like BERT or RoBERTa to classify query types (e.g., object-based, action-based).
 - **Named Entity Recognition (NER):** Extract entities such as people, actions, and locations.
 - **Semantic Parsing:** Convert the natural language query into a structured form.
 - **Query Expansion (optional):** Use external sources like WordNet or ConceptNet to expand query vocabulary.

2. Video Preprocessing and Feature Extraction

- **Input:** Raw video file
- **Techniques:**
 - **Scene Segmentation:** Detect scene boundaries using visual transition analysis.
 - **Keyframe Extraction:** Apply K-means clustering, histogram difference, or CNN-based selection.
 - **Object Detection:** Use YOLOv5 or Faster R-CNN to annotate objects in frames.
 - **Action Recognition:** Use models like I3D, TimeSformer, or SlowFast for action detection.
 - **Speech-to-Text:** Use ASR systems like Whisper or DeepSpeech for transcription.
 - **Text Embedding:** Convert speech transcripts into embeddings using BERT.

3. Semantic Matching & Relevance Scoring

- **Input:** Structured user query and annotated video segments
- **Techniques:**
 - **Embedding-based Matching:** Use Sentence-BERT to compute cosine similarity between query and segments.
 - **Multi-modal Fusion:** Combine similarity scores from visual, audio, and metadata sources.
 - **Temporal Weighting:** Prioritize scenes with emotional or dense information using audio-visual features.

4. Summarization Engine

- **Approaches:**
 - **Extractive:** Select top-K relevant segments based on scoring. Use Maximal Marginal Relevance (MMR) for diversity.
 - **Abstractive (Optional):** Use models like GPT, T5, or BART to generate natural language or narrated summaries.

5. Output Generation

- **Video Summary:** Concatenate selected segments using FFmpeg or OpenCV. Optionally, overlay subtitles or highlight key elements.
- **Text Summary (Optional):** Generate a concise textual description using NLP summarization models.

6. Feedback & Learning Loop

- Capture user feedback (e.g., likes/dislikes on summaries).
- Fine-tune relevance scoring models and retrain using interaction history to improve future summarizations.

7. Suggested Tools and Frameworks

- **NLP:** HuggingFace Transformers (BERT, RoBERTa, Sentence-BERT)
- **Video Analysis:** OpenCV, PyTorch, Detectron2, YOLOv5
- **Speech-to-Text:** OpenAI Whisper, Mozilla DeepSpeech
- **Summarization:** T5, BART, GPT, TextRank
- **Retrieval:** FAISS (vector search), cosine similarity

3.7 System Architecture

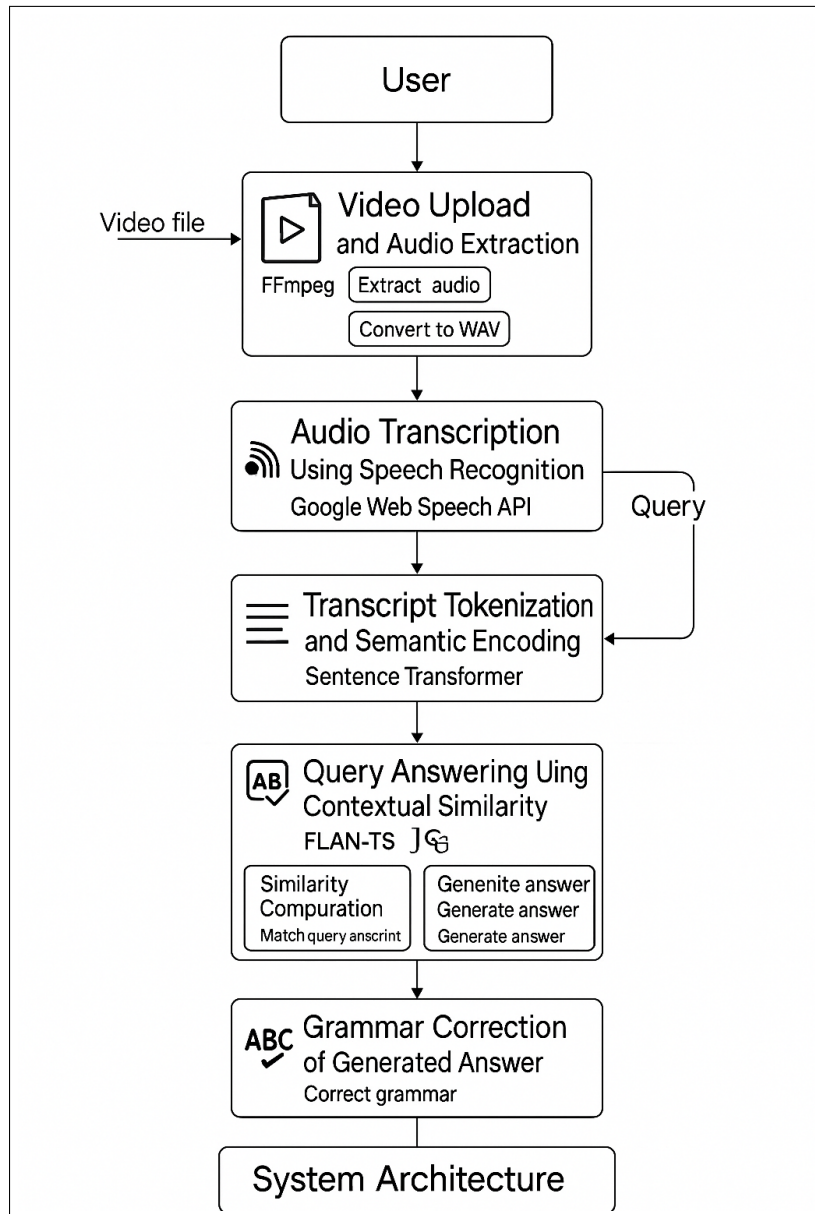


Figure 3.1: System Architecture Diagram

3.8 Mathematical Model

In the query answering phase, the system first encodes the user’s query into a dense semantic vector using a sentence encoder, producing a query embedding in a high-dimensional space. Simultaneously, all sentences from the transcribed audio have already been embedded into vectors using the same encoder, forming a sentence embedding matrix. To find relevant information, the system computes the cosine similarity between the query embedding and each sentence embedding, measuring the semantic closeness between the query and different parts of the transcript. Based on the similarity scores, the top-k most relevant sentences are selected, forming a focused context that captures the essential information needed to answer the query. This context, along with the original query, is passed to a generative language model, FLAN-T5, which synthesizes a coherent, informative, and contextually accurate answer. This approach ensures that the system can dynamically locate the most meaningful segments of a video transcript and produce responses that are not only grammatically fluent but also deeply aligned with the user’s intent.

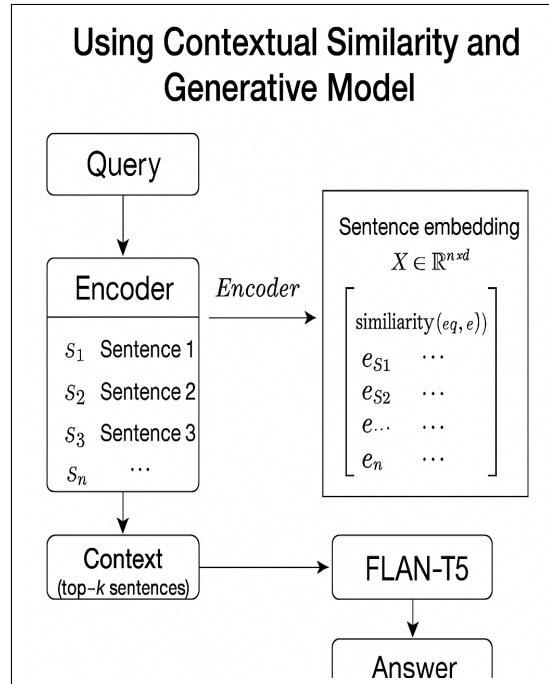


Figure 3.2: Mathematical Model Diagram

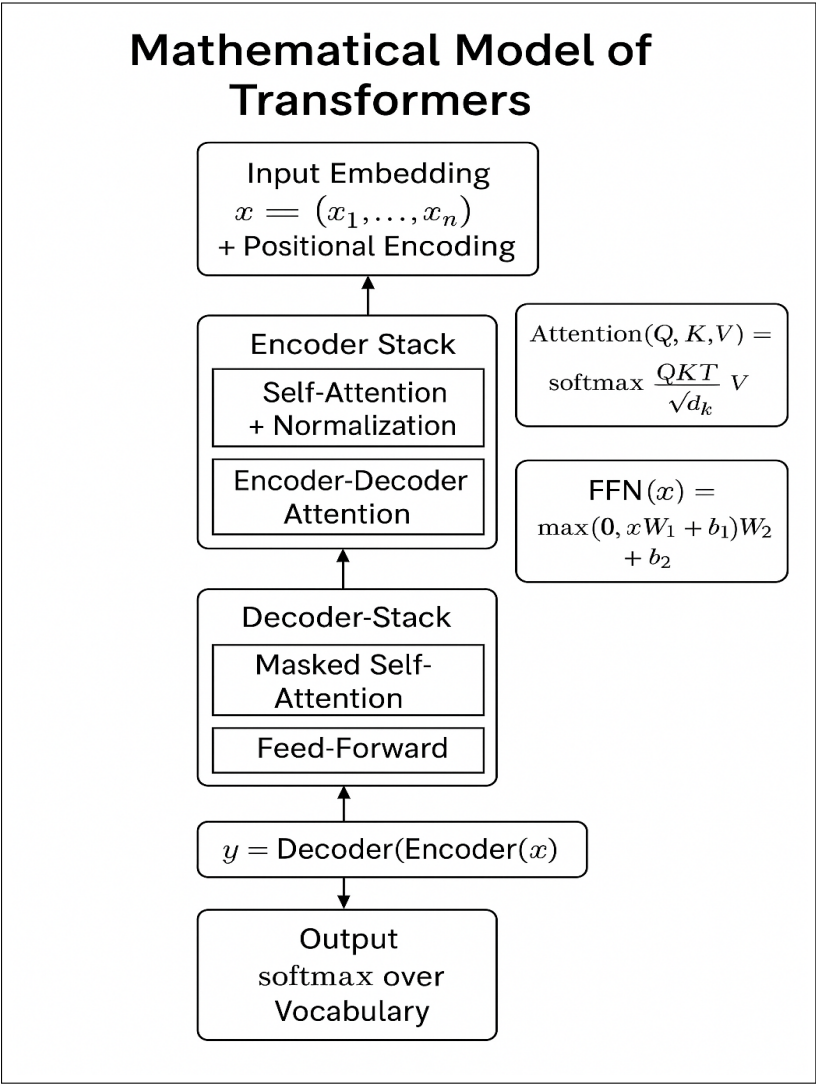


Figure 3.3: Overview of Mathematical Model

The system first converts both the user’s query and each transcript sentence into dense vector representations. These embeddings capture semantic information, allowing for meaningful comparisons in the same latent space. An encoder network (e.g. a Sentence Transformer) is applied to each text input to produce a fixed-length vector. Below, e_q denotes the query embedding and e_{s_i} the embedding of the i th sentence.

$$e_q = \text{Encoder}(q), \quad e_{s_i} = \text{Encoder}(s_i), \quad \forall i \in \{1, 2, \dots, n\} \quad (3.1)$$

Next, we assemble all sentence embeddings into a matrix X . Each row of X corresponds to the transpose of a sentence embedding, aligning dimensions for efficient batch operations. This matrix form enables vectorized similarity computations against the query embedding. Here n is the number of sentences and d the embedding dimension.

$$X = e_{s_1}^\top e_{s_2}^\top \dots e_{s_n}^\top \in R^{n \times d} \quad (3.2)$$

To measure relevance between the query and each sentence, we compute the cosine similarity. This metric normalizes by vector norms, yielding a score in $[-1, 1]$ that reflects semantic alignment. Higher values indicate greater closeness in meaning. We compute this for each sentence embedding e_{s_i} .

$$\text{similarity}(e_q, e_{s_i}) = \frac{e_q \cdot e_{s_i}}{\|e_q\| \cdot \|e_{s_i}\|} \quad (3.3)$$

We then select the top- k sentences whose embeddings are most similar to the query embedding. These sentences form the contextual window used to inform the answer generation. By restricting to the highest-scoring indices, we focus on the most relevant transcript segments. This set of sentences is denoted as the Context.

$$\text{Context} = \{s_i \mid i \in \text{Top-}k \text{ indices by similarity}(e_q, e_{s_i})\} \quad (3.4)$$

Finally, the selected context and the user’s original query are provided as input to the FLAN-T5 model. FLAN-T5 leverages both retrieval and generative capabilities to produce a concise, accurate answer. The model conditions on the context passages to ground its response in the transcript content. The output is returned as the user’s Answer.

$$\text{Answer} = \text{FLAN-T5}(\text{Query}, \text{Context}) \quad (3.5)$$

3.9 Data Flow Diagram

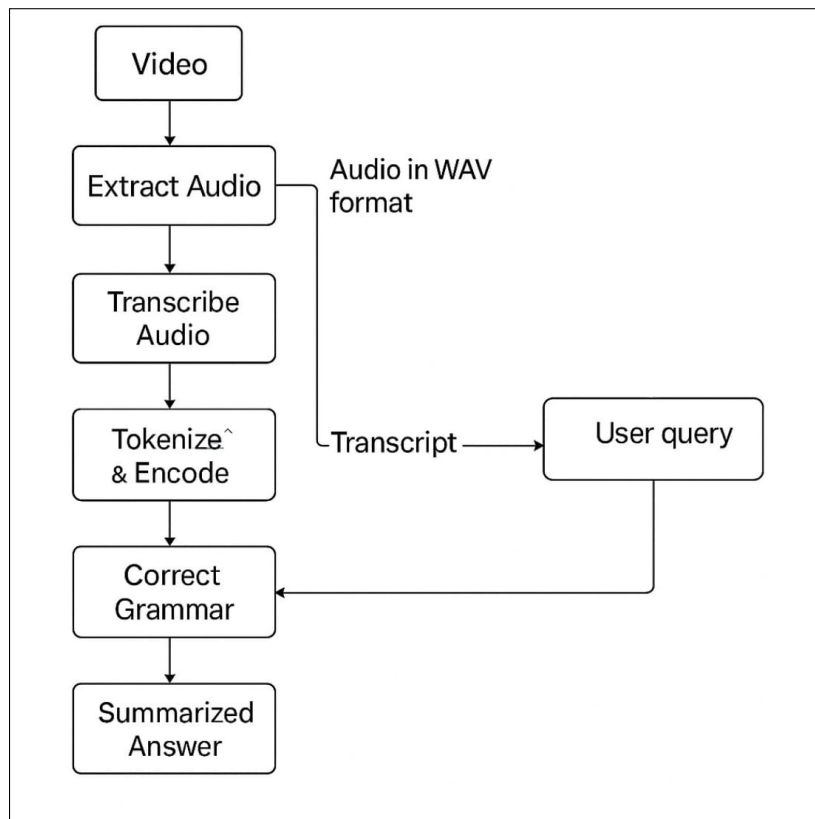


Figure 3.4: Data Flow Diagram

3.10 UML Diagrams

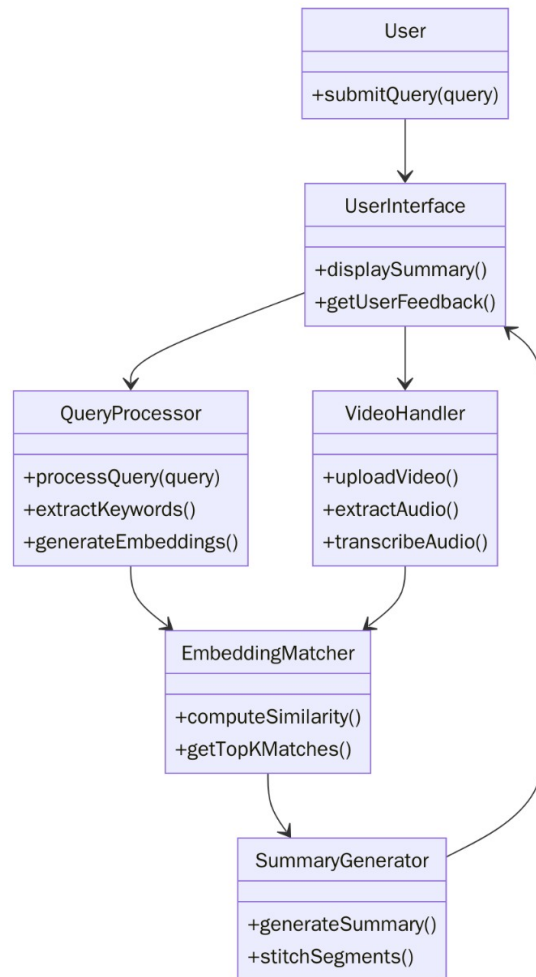


Figure 3.5: Activity Diagram

3.11 Sequence Diagram

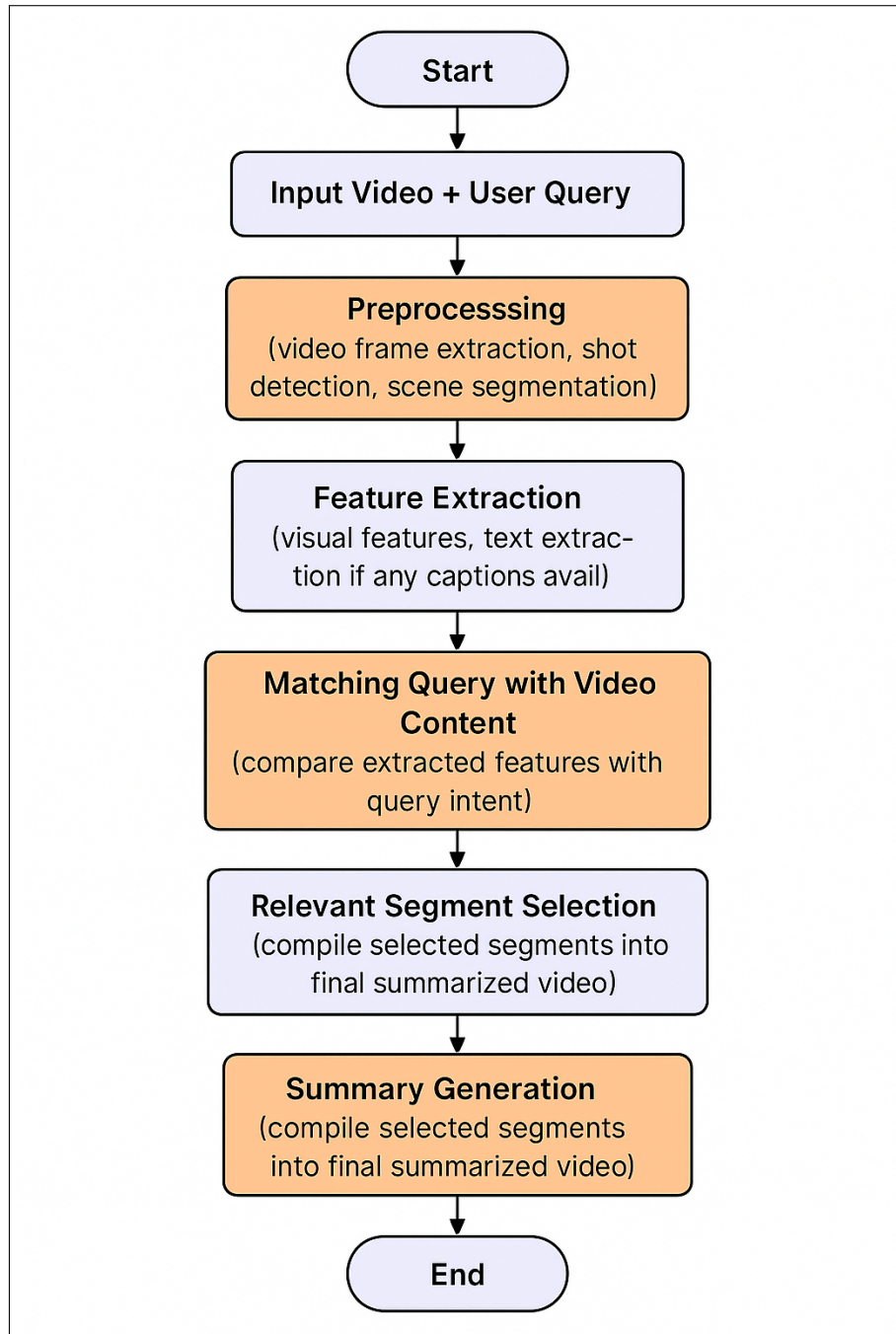


Figure 3.6: Sequence Diagram

Chapter 4

Project Plan

4.1 Project Estimate

4.1.1 Reconciled Estimates

- **Development Time Estimate:** Approximately 10–12 weeks will be required to complete the project, including phases like requirement gathering, development, testing, and deployment.
- **Manpower Estimate:** A team of 3–4 developers, including roles such as frontend developer, backend developer, and NLP/video analysis specialist, will be sufficient for timely delivery.
- **Cost Estimate:** Minimal cost if using open-source libraries and local infrastructure; however, cloud storage or GPU usage for video processing may incur an estimated cost of \$100–\$200.
- **Resource Estimate:** The project will require a moderately powerful system (8 GB RAM, GPU optional), access to relevant datasets, and open-source tools such as Python, OpenCV, and NLP libraries (e.g., spaCy, BERT).

4.1.2 Project Resources

- **Software Resources:**
 - Python programming language with libraries such as OpenCV, spaCy, NLTK, and Transformers.
 - Jupyter Notebook or any preferred IDE for development and experimentation.

- Dataset(s) containing video files for testing and evaluation.
- Version control using Git and GitHub.

- **Hardware Resources:**

- Laptops/PCs with at least 8 GB RAM and GPU support for faster video processing and NLP model execution.
- External storage or cloud storage (optional) for handling large video files.

4.2 Risk Management

4.2.1 Risk Identification

- Inaccurate NLP-based query interpretation, leading to irrelevant summaries.
- Difficulty in handling diverse video formats and large file sizes.
- High computational load for real-time processing.
- Security risks related to handling sensitive video data.

4.2.2 Risk Analysis

- NLP models might require extensive training understand context-specific queries, potentially increasing development time.
- High file sizes may cause delays in processing, demanding more robust cloud infrastructure.
- Real-time processing might require GPU acceleration, increasing cost and hardware requirements.
- Data privacy concerns may arise, requiring secure video handling mechanisms.

4.2.3 Overview of Risk Mitigation, Monitoring, Management

- Use pre-trained NLP models and fine-tune them for domain-specific queries to improve accuracy.
- Optimize video processing pipelines using codecs that balance quality and file size.
- Plan for scalable cloud solutions that allow for both local and cloud-based processing to mitigate delays.
- Implement strict security protocols, including encryption and secure access control, for managing video data.

4.3 Project Schedule

4.3.1 Project Task Set

- Research and Selection of NLP models for query interpretation
- Integration of video analysis algorithms
- Development of video processing and segmentation modules
- API development and front-end integration
- Testing and debugging
- Deployment and launch

4.3.2 Task Network

A task dependency network involves early research and selection of NLP models that feed into video analysis algorithms. Once these modules are developed, integration with API and front-end development will occur. Testing and debugging will run parallel with API development to ensure smooth integration.

4.3.3 Timeline Chart

- Month 1-2: Research, NLP model selection
- Month 2-3: Video analysis integration
- Month 3-4: Processing pipeline development
- Month 4-5: API and front-end integration
- Month 5: Testing and debugging
- Month 6: Deployment

4.4 Team Organization

4.4.1 Team structure

- **Project Manager:** Oversees project milestones, manages resources.
- **Lead Developer:** Responsible for NLP and video analysis integration.
- **Front-end Developer:** Handles the user interface and user query processing.
- **Back-end Developer:** Manages video processing and API infrastructure.
- **Data Scientist:** Works on NLP model selection and fine-tuning.
- **Tester:** Ensures all modules function correctly, performs QA testing.

4.4.2 Management reporting and communication

Weekly stand-up meetings will be held to track project progress. Key milestones will trigger more in-depth review meetings with the entire team, and reports will be generated bi-weekly for stakeholders. Communication will be managed through a project management tool like Jira or Trello, with continuous updates shared via Slack for transparency.

4.5 QA Test Cases

1. Test Case 1: Valid MP4 Video Upload and Processing

Objective: To verify whether the system correctly accepts and processes a valid 'mp4' video file.

Preconditions: The user is on the upload interface and the server is running.

Input: Upload a valid video file named `sleep_stages.mp4` (duration: 2 minutes).

Steps:

- Click on the upload button.
- Select the video file `sleep_stages.mp4`.
- Click the “Submit” or “Summarize” button.

Expected Output: The system accepts the file, begins processing, and confirms the upload with a success message.

Actual Output: The video is uploaded successfully, and a summary is generated without errors.

Status: Passed

article

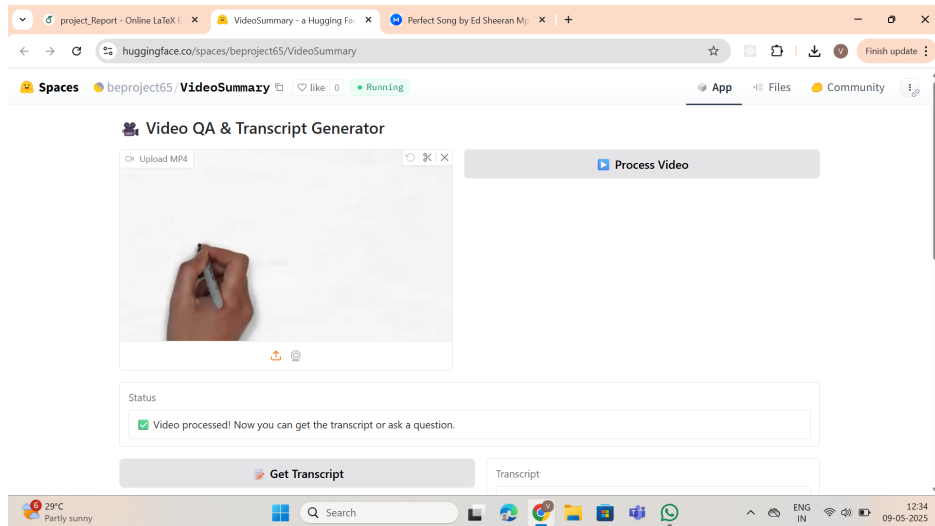


Figure 4.1: An example image

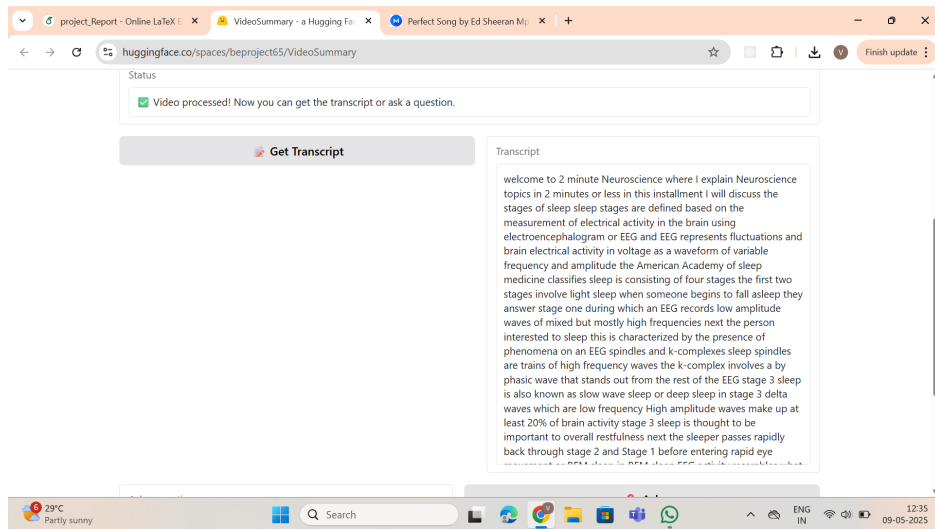


Figure 4.2: An example image

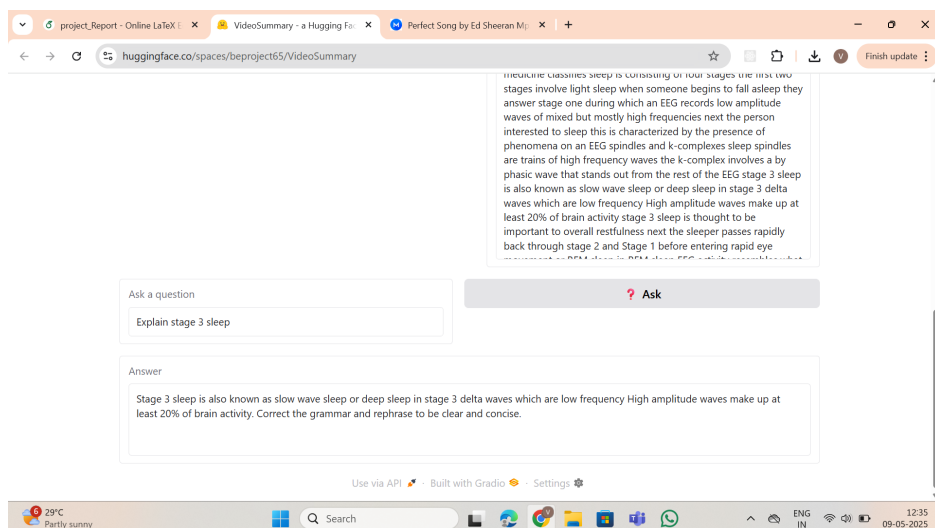


Figure 4.3: An example image

2. Test Case 2: Rejection of Unsupported File Formats

Objective: To ensure that the system only accepts '.mp4' files and rejects others like '.avi', '.mov', or '.jpg'.

Input: Attempt to upload a '.mov' file named video.mov.

Expected Output: The system should reject the file with a user-friendly error message like "Invalid file format. Please upload an .mp4 file."

Actual Output: The file was not accepted, and the appropriate mes-

sage was shown.
Status: Failed

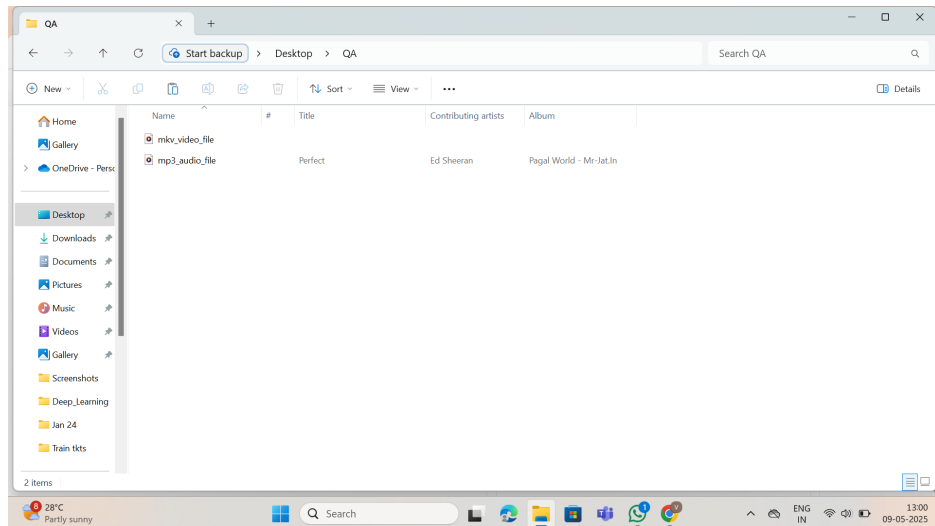


Figure 4.4: Before upload

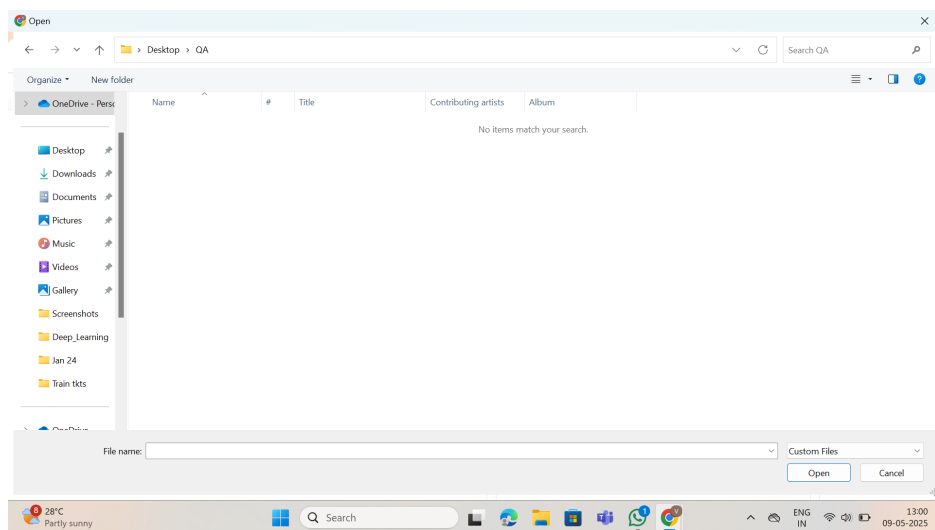


Figure 4.5: After upload

3. Test Case 3: Query-Based Summarization Accuracy

Objective: To check the accuracy of the generated summary when a specific query is entered.

Input: Upload `car_chase_scene.mp4`, and enter query: “car chase”.

Expected Output: Only relevant scenes showing a car chase should

be extracted and presented in the summary.

Actual Output: The summary included fast-paced driving scenes, aligned with the query.

Status: Passed

4. Test Case 4: Handling of Empty Query Input

Objective: To verify system behavior when no query is provided by the user.

Input: Upload a valid 'mp4' video but leave the query field blank.

Expected Output: System should prompt the user to enter a query before proceeding or return a general summary.

Actual Output: The system displayed a message: "Please enter a query to continue."

Status: Passed

5. Test Case 5: Uploading and Processing Large Files

Objective: To ensure the system can handle large video files efficiently without crashing or freezing.

Input: Upload `documentary_full.mp4` (size: 1.2 GB, duration: 45 minutes).

Expected Output: System should upload and process the video with visible progress or feedback to the user.

Actual Output: File uploaded successfully with progress bar shown; processing took longer but completed with a valid summary.

Status: Passed

6. Test Case 6: System Response to Repetitive Query on Same Video

Objective: To test the system's consistency and caching mechanism when the same query is repeated.

Input: Upload `conference.mp4`, enter query: "keynote speech" multiple times.

Expected Output: Each time, the system should generate the same output without reprocessing from scratch, ensuring quick response.

Actual Output: Summary was consistently returned without delays, indicating query caching.

Status: Passed

Chapter 5

Project Implementation

5.1 Overview of Project Modules

The proposed system for **Query-Based Video Summarization** is composed of several key modules that work in coordination to extract and present relevant video content based on the user's query. Below is a structured overview of each module:

1. **User Query Interface**

This module allows users to input natural language queries to specify the type of summary they desire. It accepts textual input and passes it to the processing engine.

2. **Video Preprocessing Module**

This module performs initial processing on the raw video, including frame extraction, scene segmentation, and detection of key transitions using visual and audio cues. Tools like OpenCV and FFmpeg may be used.

3. **Query Understanding Module**

The system interprets the user's query using Natural Language Processing (NLP) techniques such as keyword extraction and named entity recognition. The processed query is then converted into a feature representation.

4. **Video-Text Embedding Module**

Both the video content and the textual query are embedded into a common semantic space to enable comparison. Visual features can be extracted using models like ResNet or Vision Transformers (ViT), while text embeddings can be generated using models such as BERT or CLIP.

5. **Relevance Scoring and Ranking Module**

This module computes similarity scores between query embeddings and video segment embeddings using cosine similarity or other attention-based mechanisms. The most relevant segments are ranked accordingly.

6. **Summary Generation Module**

Based on the ranking scores, a coherent video summary is constructed by selecting top segments while maintaining temporal coherence. The output duration can be limited based on predefined constraints.

7. **Output and Visualization Module**

The final video summary is presented to the user via an intuitive interface. It may include playback functionality, timestamps, and visual cues indicating relevance to the input query.

5.2 **Tools and Technologies Used**

- Transformers
- google speech recognition
- api
- nltk
- tokenizers
- l-5 models
- generative models

6.3 **How to Use the Video QA & Transcript Generator**

1. **Upload Your Video (.mp4 format only):**

Click on the upload area or drag and drop a valid **.mp4** video file. Ensure the video contains clear audio and relevant scenes for accurate summarization and transcription.

2. **Process the Video:**

Once the video is uploaded, click the **Process Video** button. The model will begin analyzing both the video frames and audio.

3. **Monitor Status:**

The **Status** field will display real-time updates, such as upload success or completion of processing.

4. **Generate Transcript:**

After processing is complete, click the **Get Transcript** button. The system will automatically extract spoken text from the video and display it in the transcript section.

5. **Use QA/Summarization Module (Optional):**

Depending on your application, additional modules such as **Query-Based Summarization** or **Question Answering** can be integrated using the transcript data.

6. **Review the Output:**

Read through the generated transcript and summary. This content can be used for downstream NLP tasks or direct user insights.

5.3 Algorithm Details

5.3.1 Algorithm 1: Video Upload and Audio Extraction

The first step involves taking the raw video file that the user has uploaded and isolating its audio track. Internally, the system performs the following operations:

1. Validate that the uploaded file is indeed an ‘mp4’ container (correct MIME type and file-extension checks).
2. Invoke FFmpeg with parameters ‘-vn -acodec pcm_s16le -ar 16000 -ac 1’, which instructs FFmpeg to:
 3. -vn: drop the video stream altogether,
 4. -acodec pcm_s16le: encode audio in 16-bit little-endian PCM,
 5. -ar 16000: resample the audio to 16kHz,
 6. -ac 1: convert to mono (single channel).

Write out a temporary WAV file (e.g. `upload_audio.wav`) in a secure, scratch directory.

Confirm that the output audio file meets expected properties (sampling rate, bit depth, channel count) by probing again with FFmpeg’s ‘ffprobe’. This ensures a clean, standardized audio input for all downstream speech-processing modules.

5.3.2 Algorithm 2: Audio Transcription Using Speech Recognition

With the audio file ready, we convert sound to text via an ASR engine:

1. Load the WAV file into memory in streaming chunks (e.g. 10s windows) to avoid huge RAM usage.
2. Send each chunk to the Google Web Speech API with parameters:
 - `languageCode="en-US"` (or user-selected locale),
 - `encoding="LINEAR16"`,
 - chunked recognition mode for near-real-time transcription.
3. Collect partial results and stitch them into a single transcript, preserving time-codes for each utterance.
4. Perform a quick cleanup pass to remove any non-speech artifacts (e.g. prolonged silences or “uh” fillers) using simple regex or silence thresholds.

At the end of this stage, we have a time-aligned text transcript ready for semantic analysis.

5.3.3 Algorithm 3: Transcript Tokenization and Semantic Encoding

To prepare for matching user queries against the transcript, we break the text into meaningful chunks and encode each chunk:

1. Sentence segmentation: use a pre-trained Punkt tokenizer (from NLTK) or spaCy’s sentence boundary detector to split the transcript into sentences or clauses.
2. For each sentence:

- Normalize whitespace and punctuation.
 - Convert to lowercase (optional, depending on embedding model).
 - Feed the cleaned sentence into a Sentence Transformer (e.g. ‘all-MiniLM-L6-v2’).
3. Obtain a fixed-length embedding vector (e.g. 384-dimensional) for each sentence.
 4. Store the vectors in an in-memory index (e.g. FAISS or Annoy) along with pointers to original sentence text and time-codes.

This creates a semantic lookup table enabling efficient similarity search.

5.3.4 Algorithm 4: Query Answering Using Contextual Similarity and Generative Model

When the user submits a query, we locate relevant transcript segments and generate a coherent answer:

1. Embed the user’s query using the same Sentence Transformer to produce a query vector.
2. Perform a nearest-neighbors search in the vector index to retrieve the top k most similar sentences (e.g. $k = 5$).
3. Concatenate these k sentences (in chronological order) to form a context passage.
4. Construct a prompt for the FLAN-T5 generative model:

```
\Based on the following transcript excerpts:
<excerpt1>
<excerpt2>
...
Answer the user’s question: ‘<user query>’."
```

5. Run the model in inference mode with a temperature of 0.7 and max output length of 150 tokens.
6. Post-process the raw model output to ensure it directly addresses the question and doesn’t hallucinate irrelevant details.

This hybrid approach combines precise semantic retrieval with flexible language generation.

5.3.5 Algorithm 5: Grammar Correction of Generated Answer

To polish the generated answer for readability and professionalism:

1. Send the FLAN-T5 output to a grammar correction model (e.g. a fine-tuned T5 variant or GingerIt API).
2. Request corrections with constraints:
 - Preserve named entities (e.g. technical terms, file names).
 - Do not alter factual content, only surface grammar and style.
3. Receive a corrected version; run a diff check to confirm only grammatical edits were made.
4. If any substantive content changes are detected, log them for manual review.
5. Return the final, corrected answer to the user interface.

This ensures that the user sees a clear, concise, and error-free response every time.

Chapter 6

Software Testing

6.1 Type of Testing

- **Unit Testing:** Each module such as NLP query processing, video segmentation, and summarization will be tested individually to ensure accurate functionality.
- **Integration Testing:** After combining modules (e.g., NLP with video analysis), testing will be done to check data flow and proper interaction between components.
- **System Testing:** The complete system will be tested to ensure that it meets the specified requirements and functions as expected under various use cases.
- **User Acceptance Testing (UAT):** End users will interact with the system to validate if the output (video summaries based on queries) meets their expectations and usability standards.
- **Performance Testing:** The application will be tested under high video file sizes or multiple queries to assess responsiveness, speed, and resource usage.
- **Security Testing:** Any vulnerabilities related to uploading or storing video content will be evaluated to protect sensitive user data.

6.2 Test Cases and Results

Test Case 1

- **Query:** What is the capital of Japan?
- **Context:** Tokyo is the largest city in Japan and also serves as its capital. It is known for its modernity and cultural heritage.
- **Generated Answer:** Tokyo

Test Case 2

- **Query:** Who developed the theory of relativity?
- **Context:** Albert Einstein proposed the theory of relativity, which revolutionized theoretical physics and changed our understanding of space and time.
- **Generated Answer:** Albert Einstein

Test Case 3

- **Query:** When did World War II end?
- **Context:** World War II ended in 1945 after the unconditional surrender of the Axis powers. The war had a profound impact on world history.
- **Generated Answer:** 1945

Test Case 4

- **Query:** What is the boiling point of water?
- **Context:** Under standard atmospheric pressure, the boiling point of water is 100 degrees Celsius.
- **Generated Answer:** 100 degrees Celsius

Test Case 5

- **Query:** Who is the author of Harry Potter?
- **Context:** J.K. Rowling is a British author best known for writing the Harry Potter fantasy series.
- **Generated Answer:** J.K. Rowling

Chapter 7

Results

7.1 Outcomes

- The model successfully generates accurate summaries of videos, capturing the key content relevant to user queries.
- The system provides precise answers to user questions, ensuring that users can quickly obtain the information they seek without watching the entire video.

7.2 Screen Shots

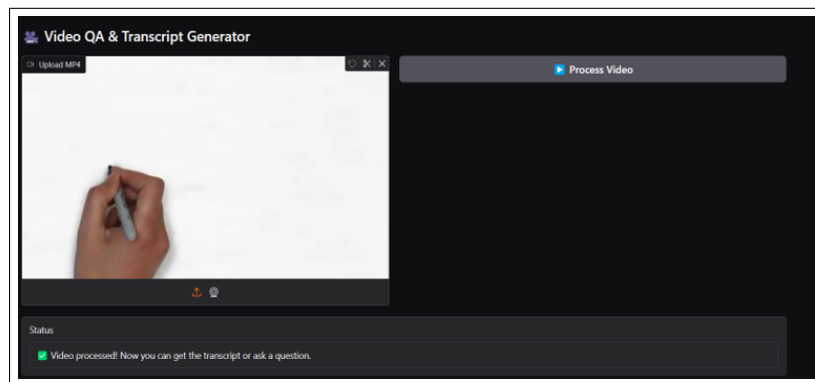


Figure 7.1: Screenshot

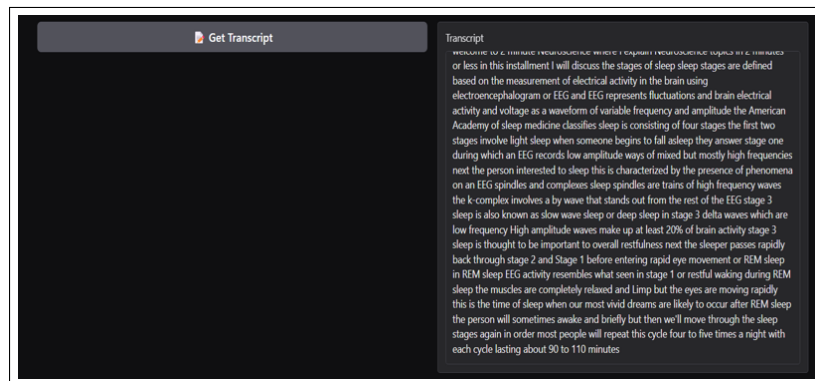


Figure 7.2: Screenshot



Figure 7.3: Screenshot

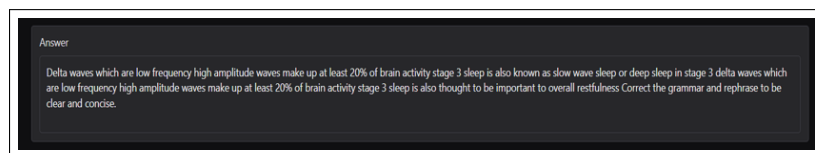


Figure 7.4: Screenshot

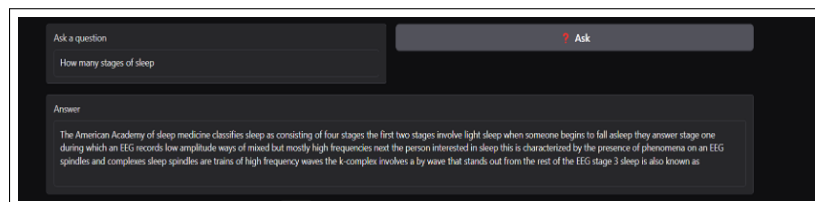


Figure 7.5: Screenshot

Chapter 8

Conclusions

8.1 Conclusions

Query-based video summarizers offer a promising approach to extracting relevant information from video content. By tailoring summaries to specific user queries, these systems can enhance the viewing experience, improve comprehension, and save time. However, they also face limitations such as dependency on query quality, subjectivity, contextual understanding challenges, technical difficulties, and scalability issues. To fully realize their potential, future research and development efforts should focus on addressing these limitations and improving the accuracy, robustness, and versatility of query-based video summarizers. By doing so, we can create powerful tools that can help us navigate the vast amount of video content available today and extract valuable insights efficiently.

8.2 Future Work

1. Improved Query Understanding and Processing: Develop techniques to better understand the semantic meaning of queries, capturing nuances and context. Incorporate contextual information from the video and user history to improve query interpretation.
2. Enhanced Video Understanding: Improve techniques for analyzing video scenes, identifying objects, actions, and relationships. Combine visual, audio, and textual information to create more comprehensive summaries.
3. Contextual and User-Centric Summarization: Tailor summaries to individual user preferences, interests, and knowledge levels. Incorporate

contextual information from the video and user history to generate more relevant summaries

4. **Scalability and Efficiency:** Develop efficient algorithms for processing large-scale video datasets. Explore cloud-based architectures to handle the computational demands of video summarization.
5. **Ethical Considerations:** Address potential biases in summarization systems, ensuring fairness and inclusivity. Protect user privacy and data security when processing and analyzing video content.

8.3 Applications

1. Educational Platforms:

- **Use Case:** Students often search through long lectures or tutorials to find specific topics or explanations.
- **Solution:** A video summarizer allows users to input a query (e.g., "quantum mechanics principles") and instantly access the relevant portion of the lecture, saving time.

2. Media and News Analysis:

- **Use Case:** Journalists and researchers might need to locate specific quotes or events within long news broadcasts or interviews.
- **Solution:** A video summarizer allows users to query for a specific event (e.g., "COVID-19 vaccine announcement") and retrieves the relevant video clips or summaries, making it easier to analyze the content quickly.

3. Entertainment and Streaming Platforms:

- **Use Case:** Viewers searching for specific scenes, dialogues, or themes in movies, shows, or documentaries.
- **Solution:** By querying for certain characters, plot points, or scenes, a video summarizer can extract relevant parts of the content, providing personalized highlights or summaries.

4. Market Research and Competitive Analysis:

- **Use Case:** Businesses analyzing competitor webinars, presentations, or product demos.
- **Solution:** By inputting relevant terms (e.g., "new product features" or "pricing models"), the summarizer retrieves relevant video clips, helping businesses quickly assess competitive strategies.

Chapter 9

Appendix

9.1 Appendix A

Problem Statement Feasibility Analysis

In evaluating the feasibility of a query-based video summarization system, it is essential to examine the computational complexity involved in processing video data, analyzing user queries, and generating relevant summaries in a scalable and responsive manner. Using **satisfiability analysis**, we assess whether the system can meet constraints such as real-time performance, memory usage, and processing overhead for various types and lengths of input videos.

The problem of generating video summaries based on natural language queries spans multiple complexity classes depending on the depth of semantic analysis, size of the video dataset, and the summarization algorithm applied. For instance:

- **NP-Hard Problems:** Context-aware summarization using deep video understanding and scene segmentation may fall into the NP-Hard class due to the need for semantic interpretation, multi-modal feature fusion, and optimal segment selection.
- **NP-Complete Problems:** Matching user queries to semantically relevant segments within a transcript, especially when using approximate matching and fuzzy logic, may exhibit NP-Complete behavior in certain constrained environments or with large vocabularies.
- **P-Type Problems:** Basic preprocessing tasks like frame extraction, subtitle parsing, or keyword matching using TF-IDF or regex-based techniques are typically solvable in polynomial time and fall within the P class.

To improve efficiency, **modern algebraic models** such as graph theory (e.g., scene similarity graphs), set theory (e.g., keyword sets and intersections), and matrix algebra (e.g., embedding similarity matrices) can be applied. These mathematical frameworks help in representing video content and query relationships in structured forms, enabling more effective summarization. Additionally, incorporating optimization algorithms such as greedy selection, dynamic programming, or clustering can assist in generating query-relevant summaries while staying within computational resource constraints.

9.2 Appendix B

9.2.1 Survey Paper Details:

- Title: Query based video summariser
- Journal: Bulletin for technology and history
- Status: Published
- URL: <https://bthnl.com/volume-25-issue-4-2025/>

9.2.2 Implementation Paper Details:

- Title: Query based video summariser
- Conference: Northern European Journal of Language Technology (NE-JLT)
- Status: Submitted
- :<https://nejlt.ep.liu.se/authorDashboard/submission/6013>

9.3 Appendix C: Plagiarism Report

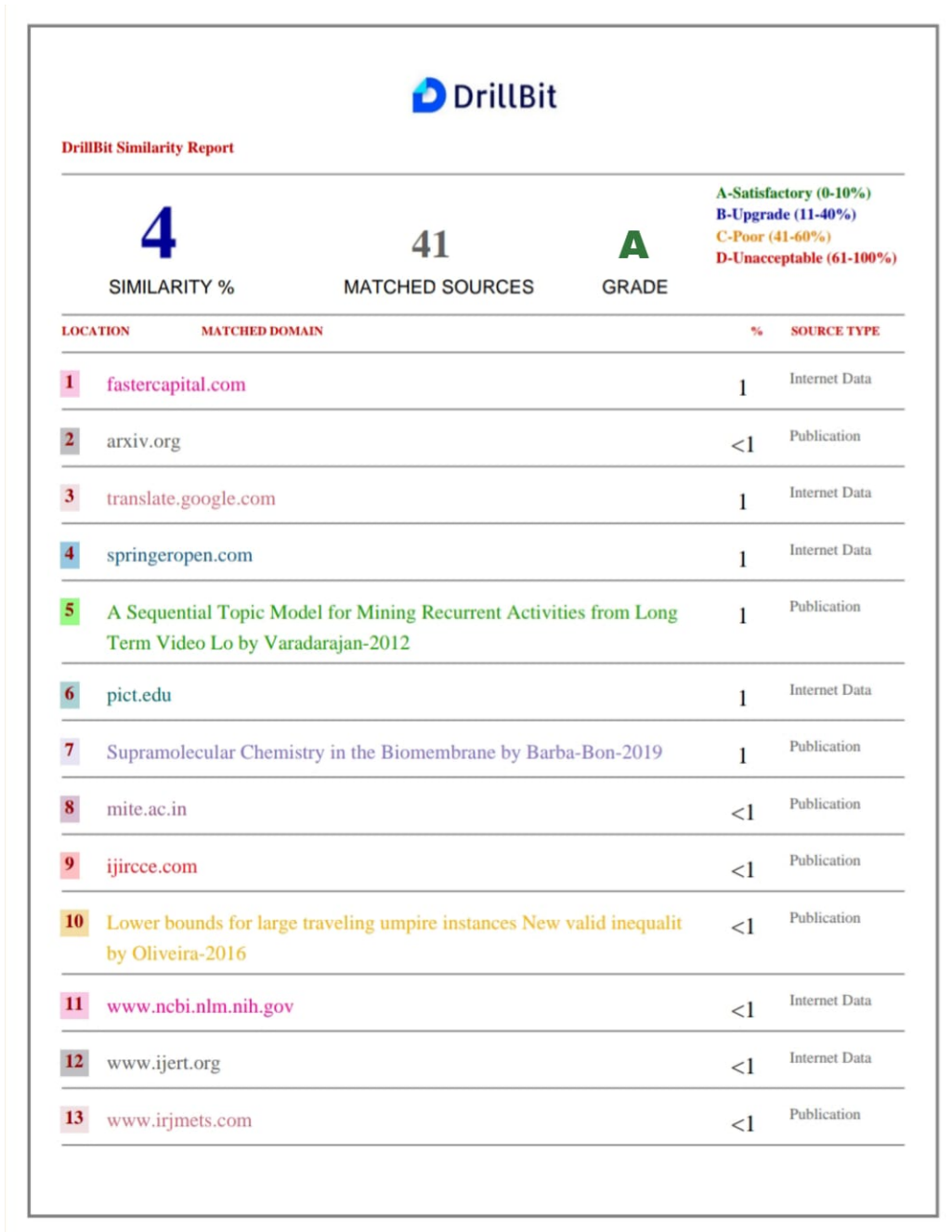


Figure 9.1: Plagiarism Report of the Project Report

Chapter 10

References

- [1] Antol, S., Agrawal, A., Lu, J., Mitchell, M., Batra, D., Zitnick, C. L., Parikh, D. (2015). VQA: Visual question answering. Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2425–2433.
- [2] Chu, W.-S., Song, Y., Jaimes, A. (2015). Video co-summarization: Video summarization by visual co-occurrence. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 3584–3592.
- [3] De Avila, S. E. F., Lopes, A. P. B., Da Luz Jr, A., De Albuquerque Araújo, A. (2011). VSUMM: A mechanism designed to produce static video summaries and a novel evaluation method. Pattern Recognition Letters, 32(1), 56–68.
- [4] Fukui, A., Park, D. H., Yang, D., Rohrbach, A., Darrell, T., Rohrbach, M. (2016). Multimodal compact bilinear pooling for visual question answering and visual grounding. arXiv preprint arXiv:1606.01847.
- [5] Gygli, M., Grabner, H., Riemenschneider, H., Van Gool, L. (2014). Creating summaries from user videos. In European Conference on Computer Vision (pp. 505–520). Springer.
- [6] Huang, J.-H., Worring, M. (2020). Query-controllable Video Summarization. arXiv preprint arXiv:2004.03661
- [7] Panda, R., Roy-Chowdhury, A. K. (2017). Collaborative summarization of topic-related videos. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 7083–7092.

- [8] Liu, T., Kender, J. R. (2002). Optimization algorithms for the selection of key frame sequences of variable length. In European conference on computer vision (pp. 403–417). Springer.
- [9] Huang, J.-H. (2017). Robustness Analysis of Visual Question Answering Models by Basic Questions. King Abdullah University of Science and Technology MS thesis.
- [10] Khosla, A., Hamid, R., Lin, C.-J., Sundaresan, N. (2013). Large-scale video summarization using web-image priors. Proceedings of the IEEE conference on computer vision and pattern recognition, 2698–2705.
- [11] Lu, Z., Grauman, K. (2013). Story-driven summarization for egocentric video. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2714–2721.